BTI 3021: Networking Project - Sprint 2

Christian Grothoff

1 Introduction

For this sprint you will write a precursor to an IP router. This precursor system is to realize the ARP protocol functionality of an IP device.

While the driver and skeleton you are given is written in C, you may use *any* language of your choice for the implementation (as long as you extend the `Makefile` with adequate build rules). However, if you choose a different language, be prepared to write additional boilerplate yourselves.

How the ARP protocol works is expected to be understood from the networking class. If not, you can find plenty of documentation and specifications on the Internet.

The basic setup is the same as in the first sprint.

1.1 Deliverables

There will be four main deliverables for the sprint:

Documentation You must document the main data structure (ARP table) and a **comprehensive test plan**.

Implementation You must implement the ARP protocol. Your implementation must answer to ARP requests, and also itself have the ability to issue ARP requests and to cache ARP replies. For this, you are to extend the `arp.c` template provided (or write the entire logic from scratch in another language).

Testing You must implement and submit your own **test cases** by *pretending* to be the network driver (see below) and sending ARP requests or command-line inputs to your program and verifying that it outputs the correct frames. Additionally, you should perform and **document** *interoperability* tests against existing implementations (i.e. other notebooks from your team to ensure that your ARP protocol implementation integrates correctly with other implementations).

All deliverables must be submitted to Git (master branch) by **December 18th 2020, 18:00 CET**.

1.2 Team

You are expected to work in a team of four students. The suggested division of work is:

1 Student Scrum master, also responsible for the documentation

1 Student Implementation

2 Students Testing

1.3 Grading

You get points for each of the key deliverables:

Technical documentation (A&D, test plan)	2
Correct implementation	8
Comprehensive test cases	3
Quality of English in documentation	2
Total	15

Parial points are awarded.

You can earn 16, 15 and 19 points in the three networking sprints, for a total of 50 points. You need **37 points** to pass the networking component of BTI 3021.

2 Detailed interface specification

The goal is to implement a program `arp` that:

1. Watches for ARP queries on the Ethernet link
2. Responds with ARP responses for your own IP address
3. Reads IPv4 addresses from `stdin` (in human-readable format), issues ARP queries for those IPv4 addresses and outputs the resulting MAC addresses. The interactive command syntax should be "arp *IP-ADDR IFNAME*" (i.e. each line is to be prefixed with the letters "arp ", followed by the IPv4 address and the name of the network interface). If the user just enteres "arp" without an IP address, you should output the ARP table in the format "*IP* -> *MAC* (*IFNAME*)" with one entry per line, i.e.

   ```
   10.54.25.15 -> 28:c6:3f:1a:0a:bf (eth1)
   ```

   (note the leading "0" digit in `0a`).
4. Provides an ARP cache so that it does not have to repeatedly make ARP requests to the network for MAC addresses it already knows.

Your programm should be called with the name of the interface, the IP address[1] for that interface and the network mask. Example:

```
$ network-driver eth0 eth1 - \
  arp eth0[IPV4:192.168.0.1/16] eth1[IPV4:10.0.0.3/24]
```

This means `eth0` is to be bound to 192.168.0.1 (netmask 255.255.0.0) and `eth1` uses 10.0.0.3 (netmask 255.255.255.0).

The file `arp.c` provides a starting point where the parsing of the command-line arguments and the `stdin`-interaction have been stubbed for you.

2.1 Routing

Implement `router` which routes IPv4 packets.

1. Populate your routing table from the network interface configuration given on the command-line using the same syntax as with the `arp` program.
2. Use the ARP logic to resolve the target MAC address. You MUST simply drop IP packets for destinations where you do not yet have the next hop's MAC address, but you MUST then issue the ARP request to obtain the destination's MAC instead (once per dropped IP packet).
3. Make sure to decrement the TTL field and recompute the CRC.
4. Generate ICMP messages for "no route to host" and "TTL exceeded".
5. Support the syntax `IFC[RO]=MTU` where `MTU` is the MTU for IFC. Example: `eth0=1500`. Implement and test IPv4 fragmentation (including *do not fragment*-flag support).
6. Support dynamic updates to the routing table via `stdin`. Base your commands on the `ip route` tool. For example, "route list" should output the routing table, and "route add 1.2.0.0/16 via 192.168.0.1 dev eth0" should add a route to `1.2.0.0/16` via the next hop `192.168.0.1` which should be reachable via `eth0`. Implement at least the `route list`, `route add` and `route del` commands.

You do not need to support VLANs, IP multicast or IP broadcast.

2.2 Suggested testing

Configure your router with `eth1` using 192.168.0.1/16. Configure `eth2` using 10.0.0.1/8 and `eth3` using 172.16.0.1/12. Connect your notebook as 10.0.0.2 using 10.0.0.1 as the default route. Set an MTU of 500 on `eth3`. Set a default route of 192.168.0.2 on the router.

Verify:

[1] You may support multiple IPs per network interface, using a comma-separated list of IPs and network masks, but this is not required.

- Correct forwarding?
- Correct address resultion and caching?
- Correct IP handling?
- Correct IP fragmentation?